

ThinFormer: Channel Sparse Transformer for Efficient HRW Object Detection

Wenxi Li^{3,4,*}, Kunpeng Liu^{1,2,*}, Moran Liu^{1,2}, Shuyang Liu³,
Chenyang Lyu⁵, Haozhe Lin¹, Yuchen Guo^{1,†}

¹Beijing National Research Center for Information Science and Technology, Tsinghua University,
Beijing, China,

²Department of Automation, Tsinghua University, Beijing, China,

³KLATASDS-MOE, School of Statistics, East China Normal University, Shanghai, China,

⁴Zhuoxi Lab, Hangzhou, China,

⁵AI Business, Alibaba International Digital Commerce, Hangzhou, China
wxli@sfs.ecnu.edu.cn, liukp22@mails.tsinghua.edu.cn, yuchen.w.guo@gmail.com

*Equal Contribution. †Corresponding author.

This supplementary material contains additional details of the main manuscript. Section 1 contains the complete proof of Theorem 1 in the paper. Section 2 presents additional details on the models and optimization strategies. Section 3 shows more visualization results on windows and bands selection mechanism proposed in the main manuscript. Section 4 provides the PyTorch-like code for Thinformer.

1 Proof of Theorem 1

Definition 1 (Dynamic Sparse Layer). A dynamic sparse layer is defined as $f_{dyn}(x) := [y_1^{dyn}, \dots, y_{N_T}^{dyn}]^\top$, where

$$y_i^{dyn} = x_i + m_i(x) \cdot \sigma(W_i'(x)x_i), \quad i = 1, \dots, N_T,$$

$m_i(x) \in \{0, 1\}$ is a dynamic mask and $W_i'(x) \in \mathbb{R}^{D \times D}$ are dynamic weights. σ is the activating function.

Definition 2 (Static Sparse Layer). A static sparse layer is defined as $f_{static}(x) := [y_1^{static}, \dots, y_{N_T}^{static}]^\top$, where

$$y_i^{static} = x_i + m_i \cdot \sigma(W_i x_i), \quad i = 1, \dots, N_T,$$

$m_i \in \{0, 1\}$ is a fixed mask and $W_i \in \mathbb{R}^{D \times D}$ are fixed weights. σ is the activating function.

Assumption 1 (Boundedness). There exist constants $R, B, B' > 0$, s.t. $\forall x \sim \mathcal{D}, i = 1, \dots, N_T$: (1) $\|x\|_2 \leq R$ and $\|x_i\|_2 \leq R/\sqrt{N_T}$; (2) The expected weights $\bar{W}_i = \mathbb{E}[m_i(x)W_i'(x)]/\mathbb{E}[m_i(x)]$ satisfy $\|\bar{W}_i\|_F \leq B$; (3) $\|W_i'(x)\|_F \leq B'$ when $m_i(x) = 1$.

Assumption 2 (Concentration). $\forall i = 1, \dots, N_T$, $m_i \sim \text{Ber}(p_i)$ and $W_i'(x)$ follows a sub-Gaussian distribution.

Assumption 3 (Activating Function). σ is L_σ -Lipschitz with $\sigma(0) = 0$.

Theorem 1. Under Assumption 1,2,3, given a dynamic sparse layer f_{dyn} , $\exists \epsilon_0 > 0, \forall \epsilon > 0$, there exists a static sparse layer f_{static} such that with probability at least $1 - \delta(\epsilon)$:

$$\|f_{static}(x) - f_{dyn}(x)\| \leq \epsilon_0 + \epsilon$$

where $\delta(\epsilon) = O(\exp(-c \cdot \epsilon^2))$ for some constant $c > 0$.

Proof. Let $\epsilon > 0$ and f_{dyn} be given, we construct a static sparse layer to approximate it and bound the error.

Fix $\tau > 0$, the value of which will be chosen later. Define $S_\tau := \{i : p_i > \tau\}$, $m_i := \mathbb{I}_{S_\tau}$, $\bar{W}_i := \bar{W}_i$.

Suppose $W_i'(x) - \bar{W}_i(x)$ is zero-mean sub-Gaussian random vector with sub-Gaussian norm σ_i (Assumption 2), i.e. $\forall t > 0$,

$$\mathbb{P}(\|W_i'(x) - \bar{W}_i(x)\|_F \geq t) \leq 2D^2 \exp(-\frac{t^2}{2D^2\sigma_i^2})$$

Taking union bound over N_T blocks, we have with probability at least $1 - \delta$,

$$\|W_i'(x) - \bar{W}_i(x)\|_F \leq \sigma_m D \sqrt{2 \log(2N_T D^2 / \delta)} =: g(\delta), \forall i \quad (1)$$

where $\sigma_m := \max \sigma_i$.

By Hoeffding Inequality and a similar union bound argument, with probability at least $1 - \delta$,

$$|m_i(x) - p_i| \leq \sqrt{\log(2N_T / \delta) / 2}, \forall i \quad (2)$$

Define the approximation error $e_i := \|y_i^{static} - y_i^{dyn}\|$. We compute it in two cases.

If $i \in S_\tau$,

$$\begin{aligned} e_i &= \|\sigma(\bar{W}_i x_i) - m_i(x) \sigma(W_i'(x) x_i)\| \\ &\leq \|\sigma(\bar{W}_i x_i) - \sigma(W_i'(x) x_i)\| \cdot m_i(x) + \|\sigma(\bar{W}_i x_i)\| \cdot |1 - m_i(x)| \\ &\leq L_\sigma \|\bar{W}_i - W_i'(x)\|_F \|x_i\| \cdot m_i(x) + L_\sigma \|\bar{W}_i\|_F \|x_i\| \cdot |1 - m_i(x)| \\ &\leq L_\sigma \|x_i\| (\|\bar{W}_i - W_i'(x)\|_F \cdot m_i(x) + B \cdot |1 - m_i(x)|) \end{aligned}$$

Using (2) and (1), and noticing that

$$|1 - m_i(x)| \leq |m_i(x) - p_i| + |p_i - 1| \leq \sqrt{\log(2N_T / \delta) / 2} + (1 - \tau),$$

we have

$$\begin{aligned} e_i &\leq \frac{L_\sigma R}{\sqrt{N_T}} \left(g(\delta) + B \left(\sqrt{\log(2N_T / \delta) / 2} + (1 - \tau) \right) \right) \\ &\leq \frac{L_\sigma R}{\sqrt{N_T}} \left(g(\delta) + B \sqrt{\log(2N_T / \delta) / 2} + B \right) \end{aligned}$$

Otherwise $i \notin S_\tau$,

$$\begin{aligned} e_i &= \|0 - m_i(x)\sigma(W'_i(x)x_i)\| = m_i(x)\|\sigma(W'_i(x)x_i)\| \\ &\leq m_i(x)L_\sigma\|W'_i(x)\|_F\|x_i\| \leq m_i(x)L_\sigma B'\|x_i\|. \end{aligned}$$

Using (2) and noticing that $p_i < \tau$ in this case, we have

$$e_i \leq \frac{L_\sigma RB'}{\sqrt{N_T}} \left(\tau + \sqrt{\log(2N_T/\delta)/2} \right)$$

The overall approximation error is given by

$$\begin{aligned} &\|f_{\text{static}}(x) - f_{\text{dyn}}(x)\|^2 \\ &= \sum_{i=1}^{N_T} e_i^2 \\ &\leq \sum_{i \in S_\tau} \left[\frac{L_\sigma R}{\sqrt{N_T}} \left(g(\delta) + B\sqrt{\log(2N_T/\delta)/2} + B \right) \right]^2 \\ &\quad + \sum_{i \notin S_\tau} \left[\frac{L_\sigma RB'}{\sqrt{N_T}} \left(\tau + \sqrt{\log(2N_T/\delta)/2} \right) \right]^2 \end{aligned}$$

Denote $|S_\tau| =: M$, we have

$$\begin{aligned} &\|f_{\text{static}}(x) - f_{\text{dyn}}(x)\| \\ &\leq \sqrt{M} \cdot \frac{L_\sigma R}{\sqrt{N_T}} \left(g(\delta) + B\sqrt{\log(2N_T/\delta)/2} + B \right) \\ &\quad + \sqrt{N_T} \cdot \frac{L_\sigma RB'}{\sqrt{N_T}} \left(\tau + \sqrt{\log(2N_T/\delta)/2} \right) \\ &= \sqrt{M} \cdot C_1(\delta) + \sqrt{N_T} \cdot C_2(\delta) + \epsilon_0 \end{aligned}$$

where

$$\begin{aligned} C_1(\delta) &= \frac{L_\sigma R}{\sqrt{N_T}} \left(g(\delta) + B\sqrt{\log(2N_T/\delta)/2} \right) \\ C_2(\delta) &= \frac{L_\sigma RB'}{\sqrt{N_T}} \left(\tau + \sqrt{\log(2N_T/\delta)/2} \right) \\ \epsilon_0 &= \frac{L_\sigma RB}{\sqrt{N_T}} \end{aligned}$$

To control the approximation error, we may choose

$$\begin{aligned} \tau &< \frac{\epsilon}{2L_\sigma RB'} \\ \delta &= \inf\{\delta > 0 : \sqrt{M} \cdot C_1(\delta) + \sqrt{N_T} \cdot C_2(\delta) < \epsilon\} \end{aligned}$$

Since $\tau = O(\epsilon)$, $g(\delta) = O(\sqrt{\log(1/\delta)})$, one can further verify that $\delta = O(\exp(-c \cdot \epsilon^2))$, which completes the proof. $\square$

2 Details of Models and Training

2.1 Dataset Preparation

In this paper, we adopt a multi-scale pre-process strategy to obtain better performance. We crop the patches into 4 different sizes: 1500×1500 , 3000×3000 , 6000×6000 , 12000×12000 , with overlapping step of half the size. Patches with no objects are filtered out. During the training phase, all patches are resized to the identical shape. In fact, we are using the sliding window strategy on downsampled images.

2.2 Optimization Strategy

Previous work He *et al.* [2019] points out that training from scratch is no worse than fine-tuning. Thus we train Thinformer from scratch in a totally end-to-end manner. In other words, we did not delay the training of the scoring net and sparsity ratio generator. Instead, ALL parts of Thinformer are trained together.

In this paper, Thinformer is trained for 60 epochs with AdamW Loshchilov and Hutter [2019] and a cosine annealing strategy.

3 Visualization Results

To enhance further understanding of the behaviors of Thinformer, we present more visualization results as a complement to the main manuscript.

Figure 1 shows the estimated scores that guide spatial sparsification at each stage of the backbone. The model effectively learns to assign higher scores (highlighted regions) to areas containing objects, thereby preserving salient information while discarding background.

Figure 2 shows the amplitude spectra of the average feature map of the first stage within different bands and highlights the bands selected by our model (red bounding boxes). The selected bands clearly correspond to spectra with richer structural information, whereas bands with dominating low-frequency coefficients (which is, in some sense, with much redundant information) are pruned. This confirms that Thinformer learns to identify and process the most informative frequency components to some extent.

It is hard to analyze the band selection mechanism theoretically and thoroughly. One may notice that more complex spectra do not necessarily lead to higher scores. This phenomenon may be explained by the fact that the average pooling procedure might conceal the actual complexity. We hope that later research will facilitate understanding on this aspect.

4 Code in PyTorch

The core of Thinformer lies in the *global blocks* and *local blocks*. Since global MSA, FFN and local MSA are already presented in previous works, we highlight our novel *dynamic local FFN* in Algorithm 1. Local FFN enables us to utilize the inherent sparsity in channel domain, which leads to the significant reduction of GFLOPs demonstrated in the main manuscript.

The whole code will be made publicly available after the review process.

References

- Kaiming He, Ross Girshick, and Piotr Dollár. Rethinking imagenet pre-training. In *ICCV*, 2019.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019.



Figure 1: **Visualization of the scores in the spatial domain in each stage of Thinformer.** The leftmost column stands for the original shots and the following columns exhibit the scores of the feature map in each stage. Highlighted pixels represent higher scores.

Algorithm 1 Local FFN, PyTorch-like Pseudocode

```
class DynamicFFN(nn.Module):
    def __init__(self, dim):
        super().__init__()
        self.dim = dim
        self.pool = nn.AdaptiveAvgPool2d((1, 1))
        self.conv1 = nn.Conv2d(dim, dim**2, 1, bias=False)
        self.bn = nn.BatchNorm2d(dim**2)
        self.relu = nn.ReLU()
        self.conv2 = nn.Conv2d(dim**2, dim**2, 1)
        if bias:
            self.bias = nn.Parameter(torch.zeros(dim))
        else:
            self.bias = None

    def forward(self, x):
        B, C, H, W = x.shape
        weight = self.conv2(self.relu(self.bn(self.conv1(self.pool(x)))))
        weight = weight.view(B * self.dim, self.dim, 1, 1)
        x = F.conv2d(x.reshape(1, -1, h, w), weight, self.bias.repeat(b), stride=1, groups=B)
        x = x.view(B, C, H, W)
        return x
```

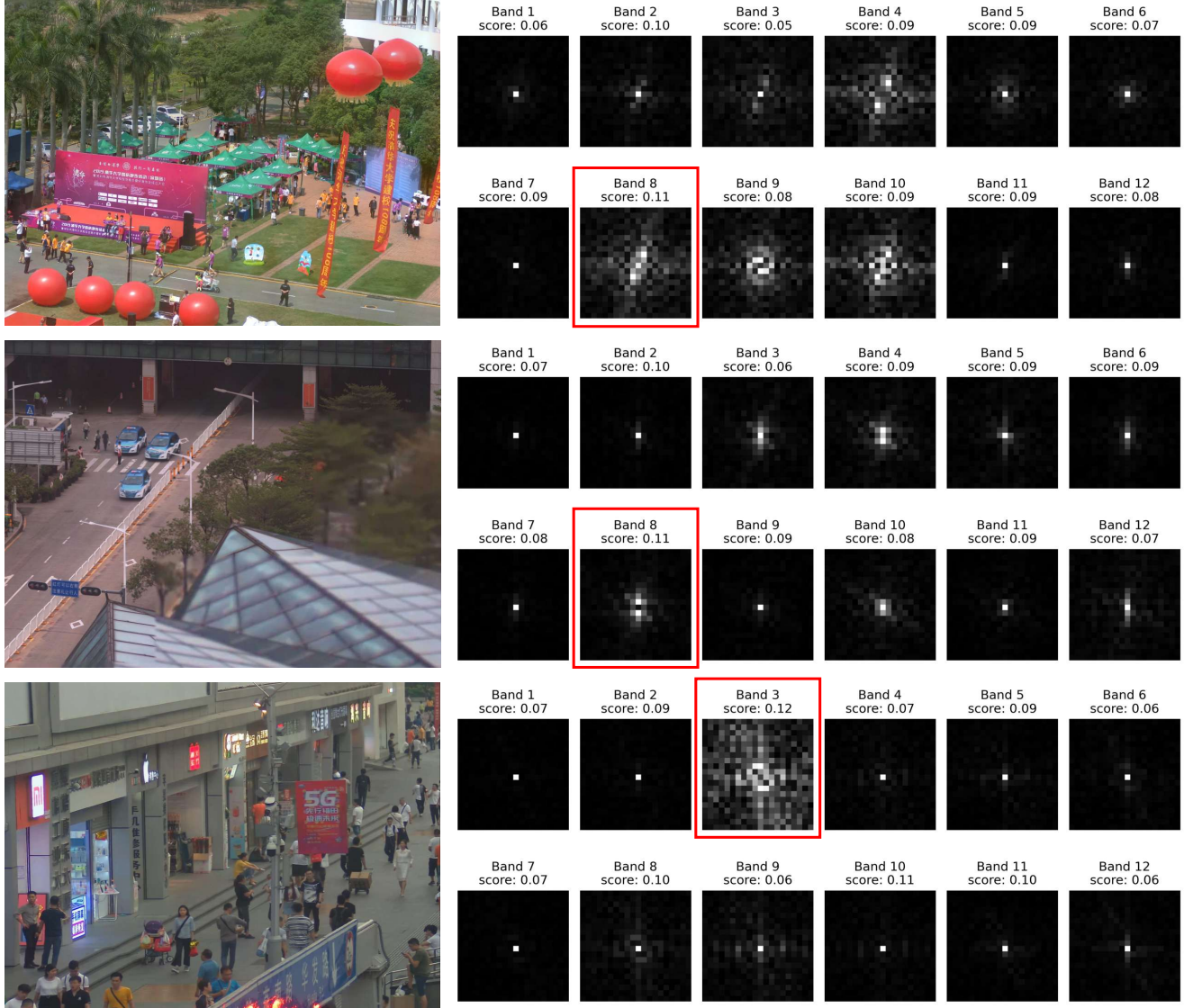


Figure 2: **Visualization of the scores in the channel domain.** We record the score allocated to each band in stage 1 and perform Fourier Transform on the average feature map within each band. Presented are the obtained amplitude spectra. The selected bands are highlighted using red bounding boxes. The leftmost picture stands for the original shot.